U. V. Patel College of Engineering, Ganpat University

Practice Questions

2CEIT702 – Big Data Analytics | Second Internal Examination – September 2026

10 practice questions | 5 marks each

Section A Apache Kafka

Q1. **[5 Marks]**

Explain Kafka's data model.

Points to cover: message, topic, producer, consumer, broker, cluster; how the pieces fit together; one example such as a `logs` *topic.*

Q2. **[5 Marks]**

Explain the anatomy of a Kafka message, describing every part it carries.

Points to cover: key, value, compression type, headers, partition and offset, timestamp; which parts are optional.

Q3. **[5 Marks]**

What is an offset in Apache Kafka? Explain how topics are divided into partitions and what ordering Kafka does and does not guarantee.

Points to cover: definition of an offset, numbering from 0, offsets are unique inside one partition only, order guaranteed within a partition but not across partitions.

Q4. **[5 Marks]**

Explain Kafka consumer groups and consumer offsets. A topic has 4 partitions. State what happens when the group has 2 consumers and what happens when the group has 6 consumers.

Points to cover: purpose of a consumer group, one partition to one member, `group.id`*, the* `__consumer_offsets` *topic; 2 consumers share the 4 partitions, 6 consumers leave 2 idle.*

Q5. **[5 Marks]**

What is a poison pill in Apache Kafka? Explain why it occurs and how an application should deal with it.

Points to cover: definition, the serialization and deserialization rule, why the consumer loops on the same offset, retry then quarantine to a separate topic, the broker's limited role.

Section B Apache Cassandra

Q6. **[5 Marks]**
Explain the data model of Apache Cassandra, starting from the column and ending at the cluster.

Points to cover: column, row, partition, table, keyspace, cluster; composite primary key; jobs of the partition key and the clustering columns.

Q7. **[5 Marks]**
Explain the main characteristics of Apache Cassandra.

Points to cover: distributed, decentralized, elastically scalable, highly available and fault tolerant, tuneably consistent, row oriented; the peer-to-peer ring and gossip; no single point of failure.

Q8. **[5 Marks]**
State Brewer's CAP theorem and explain how the consistency level and the replication factor work together in Cassandra.

Points to cover: the three properties and the rule that only two can be strongly supported, where Cassandra stands, replication factor set on the keyspace, consistency level set by the client on every query, what happens when the consistency level is lower than the replication factor.

Section C MapReduce, YARN and Apache ZooKeeper

Q9. **[5 Marks]**
Explain the MapReduce algorithm using the word count example, and state two differences between Hadoop 1.x MapReduce and YARN.

Points to cover: map, shuffle and reduce with the key and value pairs at each step; JobTracker and TaskTracker against Resource Manager, Node Manager and Application Master; fixed slots against containers.

Q10. **[5 Marks]**
Explain Apache ZooKeeper. Describe the types of znode and state how a leader is elected in an ensemble.

Points to cover: use of ZooKeeper, the znode namespace, persistent, ephemeral and sequential znodes, the ensemble and the majority rule, election using ephemeral sequential znodes.

Answer Outlines

Each outline lists the points an examiner expects in a five-mark answer. Write them in your own words and add the example or the diagram.

Q1 Kafka's data model. The Kafka data model consists of messages and topics. A **message** represents a piece of information such as a line in a log file, a row of stock market data or an error message from a system. Messages are grouped into categories called **topics**, for example a topic named `logs` and a topic named `purchases`, and a topic is identified by its name. The data in a topic is stored in key and value form in binary format. The processes that publish messages into a topic are called **producers**, and the processes that receive messages from a topic are called **consumers**. The processes or servers that store and process the messages are called **brokers**, and a set of brokers working together forms a **Kafka cluster**. Each topic is further divided into **partitions**, and every message written into a partition receives an **offset**. Unlike SQL tables, Kafka topics are not query-able; data is written by producers and read in order by consumers.

Q2 Anatomy of a Kafka message. A Kafka message carries the following parts. The **key** is optional and may be null; it may be a string, a number or any object and is serialized into binary form, and it decides the partition a message goes to. The **value** is the content of the message; its format is arbitrary, it is also serialized into binary and it may be null as well. The **compression type** may be specified for the message, the options being none, gzip, lz4, snappy and zstd. The **headers** are an optional list of key and value pairs used to hold metadata about the message, most commonly for tracing. The **partition and offset** are assigned once the message is written into the topic, and the combination of topic, partition and offset identifies the message uniquely. Finally a **timestamp** is added either by the user or by the system.

Q3 Offsets, partitions and ordering. A topic is broken down into a number of **partitions**, and the number is fixed when the topic is created. Partitions are numbered from 0 to N minus 1, so a topic with 5 partitions has the partitions 0, 1, 2, 3 and 4. An **offset** is an integer that Kafka adds to each message as it is written into a partition, and every message in a given partition has a unique offset; the numbering begins at 0 and increases for every new message. Offsets have a meaning only for a specific partition, so offset 3 of partition 0 and offset 3 of partition 1 are two different messages. Old messages are deleted over time but the offsets are never re-used. Kafka guarantees the **order of messages within a partition**; it gives **no ordering across partitions**, because a consumer reads several partitions at the same time. Messages are also immutable, so once written they cannot be changed.

Q4 Consumer groups and consumer offsets. Consumers that belong to the same application and perform the same logical job are grouped into a **consumer group**, which is indicated by setting the same `group.id`. The members of a group coordinate through a group coordinator and split the work of reading from the different partitions, so that one partition is assigned to only one member of that group at a time; partitions are therefore the unit of parallelism. To remember how far it has read, the group commits the position of the last successfully processed message, called the **consumer offset**, into the internal topic

`__consumer_offsets`, and this acts as a checkpoint after a restart. For a topic with 4 partitions: with **2 consumers** in the group, each consumer is assigned 2 partitions and both are busy; with **6 consumers**, four consumers take one partition each and the remaining **2 consumers stay idle**, because a partition is never shared inside a group. More throughput is obtained by creating more partitions, not by adding more consumers.

Q5 Poison pills. Messages that are sent to a Kafka topic but do not respect the agreed serialization format are called **poison pills**. They arise because data must be deserialized in the same format in which it was serialized: a String written with StringSerializer must be read with StringDeserializer, and the format of a topic must not change during its lifecycle. Typical causes are malformed data, an incompatible schema or a bug in the application. The effect is that the consumer fails on that record, does not commit an offset beyond it, restarts and fetches the very same record again, which produces an endless loop. The correct handling is to detect and classify the failure, retry only transient failures with a bounded backoff, and then quarantine the bad record in a separate topic, often called a DLQ, together with its topic, partition, offset, key and error information, before advancing the offset by policy. The broker only stores the bytes and serves them again at the same offset; it does not understand business validity and does not create a quarantine area by itself.

Q6 Cassandra's data model. From the smallest unit upwards: the **column** is a name and value pair; the **row** is a container for columns referenced by a primary key; the **partition** is a group of related rows stored together on the same nodes; the **table** is a container for rows organised by partitions; the **keyspace** is a container for tables and corresponds closely to a database in the relational model, carrying attributes such as replication; and the **cluster**, also called the ring, is the container for keyspaces and spans one or more nodes. Cassandra uses a **composite (compound) primary key** made of a **partition key** plus an optional set of **clustering columns**. The partition key determines the nodes on which the rows are stored and may itself consist of more than one column, while the clustering columns control how the data is sorted for storage within a partition. The model is described as a partitioned row store in sparse multidimensional hash tables, where sparse means a row need not carry the same columns as the other rows. A **static column** holds a value that is outside the primary key but shared by every row of a partition.

Q7 Characteristics of Cassandra. Cassandra is an open-source, **distributed** database, meaning it runs on many machines yet appears to the user as one unified whole. It is **decentralized**, meaning every node is identical and no node performs organising operations that the others do not; the nodes form a peer-to-peer ring and use a **gossip protocol** to keep the list of live and dead nodes in sync, which removes the single point of failure and makes running fifty nodes little different from running one. It is **elastically scalable**, a special property of horizontal scalability, so the cluster can accept new nodes that start serving requests without a major reconfiguration and can also be scaled back down. It is **highly available and fault tolerant**, since failed nodes can be replaced with no downtime and data can be replicated to several data centres for local performance and for protection against a disaster. It is **tuneably consistent**, letting the client decide the level of consistency required in balance with availability. It is **row oriented**, a partitioned row store, and gives **high performance** on multiprocessor and multicore commodity machines, scaling to hundreds of

terabytes. Its distribution design comes from Amazon's Dynamo and its data model from Google's Bigtable.

Q8 CAP theorem, consistency level and replication factor. Brewer's CAP theorem names three properties: **Consistency**, meaning all clients read the same value for the same query even with concurrent updates; **Availability**, meaning all clients can always read and write; and **Partition tolerance**, meaning the database keeps functioning when the network is segmented. The theorem states that in any given system only two of the three can be strongly supported. A CA system typically uses two-phase commit and blocks when a partition occurs; a CP system stays consistent but some data may become unavailable when nodes fail; an AP system always answers but the answer may be old or wrong. Cassandra sits on the AP side and uses eventual consistency, propagating updates to replicas in the background so that it is always writable. The **replication factor** is an attribute of the keyspace and states how many nodes hold a copy of a value. The **consistency level** is specified by the client on every single query and states how many of those replicas must acknowledge a write or respond to a read. The consistency level is therefore based on the replication factor and not on the number of nodes in the cluster. Setting the consistency level equal to the replication factor gives the strongest consistency but blocks until all replicas answer; setting it lower means the operation is considered successful even if some nodes are down.

Q9 MapReduce with word count, and YARN. MapReduce is a programming model introduced by Google that performs parallel and distributed processing on huge data sets, and it works in three steps. The **map function** first splits the input data set into smaller sub-data sets and then applies the mapping computation to each of them in parallel, emitting key and value pairs such as `<the,1>`, `<data,1>`, `<the,1>`. The **shuffle function** merges all the pairs that share a key, giving `<Key, List<Value>>` such as `<the,[1,1]>`, and then sorts those pairs by key. The **reduce function** takes the sorted pairs and produces the final output, for example `<the,2>` and `<data,1>`. An optional **combiner** performs a local reduction on the map side to save network bandwidth, and a **partitioner** makes sure that all the data of one key reaches the same reduce task. Differences from YARN (any two): Hadoop 1.x uses a single **JobTracker** for resource allocation, scheduling and monitoring together with TaskTrackers, and it is a single point of failure, whereas YARN separates these into the **Resource Manager**, the **Node Manager** and a per-application **Application Master**; Hadoop 1.x has fixed map slots and reduce slots that go to waste, whereas YARN uses **containers** of memory and CPU cores in which any task may run; and Hadoop 1.x can run only MapReduce jobs, whereas YARN can also run streaming, interactive and graph processing applications.

Q10 Apache ZooKeeper. Apache ZooKeeper is a distributed, open-source coordination service that provides configuration maintenance, synchronization and a naming registry for distributed applications, and it is used by systems such as Hadoop, HBase, Storm and Kafka message engines. It keeps data in a hierarchical namespace that looks like a file system, in which every node is called a **znode** and, unlike a normal file system, every node can hold data; the data is kept in memory for high throughput and low latency, and each update is stamped with a number that reflects the order of transactions. There are three types of znode.

A **persistent znode** stays until it is deleted manually and is created by default when no flag is passed. An **ephemeral znode** is deleted automatically by ZooKeeper once the client that created it ends its session. A **sequential znode** receives a ten digit sequence number appended to its path, and it may be persistent or ephemeral; the ephemeral sequential form is used for locking and synchronization. The servers form an **ensemble**, which needs a minimum of three nodes and follows the quorum rule $Q = 2N + 1$, so a majority of more than fifty per cent must be live; otherwise the leader steps down and the service becomes read only. For **leader election**, a persistent znode `/election` is created and every joining server creates an ephemeral sequential znode `/election/leader-<n>` under it. The server holding the least sequence number becomes the leader, and every other server watches the znode with one lower sequence number, so that when the leader's znode is deleted only the next candidate is notified.